Analyse de tavelures laser par apprentissage profond
Description et planification

Projet Intégrateur

Gabriel Genest

Sous la direction de :

Luc Lamontagne

Table des matières

Table des matières ii

1 Description du problème 1

2 Revue de littérature 3

3 Choix des modèles 5
- 3.1 Modèle principal 5
- 3.2 Modèle secondaire 6
- 3.3 Pipeline d'entraînement 6

4 Données du modèle 9

5 Tâches à accomplir 13

A Base mathématique des tavelures 14

Bibliographie 15

Chapitre 1

Description du problème

Le projet présenté dans ce document est centré sur une problématique touchant l'imagerie et la microscopie. Ces domaines sont au centre de la médecine, de la biophotonique et de plusieurs procédés industriels. Alors que plusieurs cherchent à améliorer la résolution en dépassant le critère de Rayleigh, il existe une modalité d'imagerie simple et naturelle qui a gagné en popularité au cours des dernières années, malgré son émergence au cours des années 1980. Il s'agit de l'imagerie à l'aide des tavelures laser, ou encore *laser speckle imaging* en anglais. Les tavelures laser apparaissent lorsqu'un faisceau de lumière cohérente frappe une surface ayant des aspérités microscopique de sorte que la lumière est diffusée dans des directions quelconques qu'on simplifie comme aléatoires. Cette diffusion cause différents délais pour les ondes réfléchies, créant des interférences destructives ou constructives lors de la détection de la lumière. La base mathématique des tavelures est bien connue du monde de l'optique statistique (voir l'annexe A pour plus d'informations), mais une utilisation efficace et rapide des tavelures pour l'imagerie reste une étape cruciale. Il est souvent intéressant d'avoir une analyse en temps réel des tavelures, permettant de suivre l'évolution de certains matériaux ou phénomènes biologiques. Il a donc été jugé pertinent d'utiliser la puissance des architectures d'apprentissage par réseaux profonds pour automatiser et potentiellement accélérer l'analyse de tavelures, que ce soit dans le domaine spatial ou temporel.

Dans un premier temps, Il existe des méthodes d'analyses statistiques classiques, mais selon la nature stochastique des tavelures, ces méthodes changent et induisent un biais. Ces analyses sont souvent assez lourdes mathématiquement et computationnellement, demandant par exemple plusieurs images dans un film ou encore des propriétés stochastiques bien spécifiques, comme l'ergodicité. Il est possible de modifier ces méthodes si certaines conditions ne sont pas respectées, mais savoir si elles le sont est souvent une autre tâche supplémentaire apportant son lot de défis. Utiliser une approche par apprentissage profond pourrait se spécialiser selon les besoins sans avoir à changer la structure. Si les tavelures sont stationnaire dans l'espace ou non, le réseau devrait être capable de distinguer sans l'intervention humaine

lors de l'analyse.

Dans un second temps, plusieurs analyses sont spécifiques à une problématique bien particulière. Cela multiplie les modèles et algorithmes, menant à une recherche difficile pour trouver le bon moyen. Par exemple, si on est intéressé à calculer la vitesse de déplacement des éléments diffuseurs, on doit avoir un modèle qui la calcule en fonction des données statistiques contenues dans les tavelures. Si on est plutôt intéressé à calculer la rigidité du matériau, on doit utiliser un modèle différent faisant le pont entre les statistiques des tavelures et le module de visco-élasticité, par exemple. Dans ces deux cas, les modèles risquent d'être assez différents ou encore ayant une constructions difficile à implémenter. Utiliser une approche par apprentissage profond pourrait, selon l'entraînement, construire son propre modèle interne et apprendre comment extraire ces informations sans interventions humaines lors de l'analyse.

Comme le phénomène des tavelures laser est autant spatial que temporel, un réseau alliant le contenu spatial et le contenu temporel est considéré. Le réseau aura comme entrée une séquence d'images de tavelures dans le temps et devra sortir l'information pertinente voulue. Dans le cadre de ce projet, cette information est le temps caractéristique de corrélation. La tête du réseau pourrait être modifiée selon l'information à extraire, par exemple la vitesse des diffuseurs. Une architecture mixte composée d'un réseau à convolution (CNN) et d'un réseau récurrent (RNN) sera utilisé, permettant un travail indépendant dans l'espace avec le CNN et l'information du CNN sera par la suite fournie au RNN pour l'aspect temporel. Ainsi, le réseau devrait être capable de cibler les informations pertinentes dans la corrélation spatiale et celle temporelle pour développer son modèle interne.

Chapitre 2

Revue de littérature

Plusieurs articles présentent des limitations de modèles qui seront à considérer durant le développement du projet :

— Briers et al. [1] expliquent quelques problèmes des modèles fixes de l'analyse des tavelures. Par exemple, certains modèles requiet la connaissance à priori de la distribution de la vitesse des diffuseurs, ce qui est en général pas possible, mais où on peut faire certaines hypothèses. La présence de diffuseurs statiques et dynamiques change aussi l'interprétabilité, car les statistiques changent spatialement, brisant la stationarité et l'ergodicité parfois requise.

— Liu et al. [7] expliquent la multiplication des hypothèses et des modèles. Selon les hypothèses, les modèles changent et deviennent plutôt complexes. Dans cet article, on calcule le le *blood flow index* (BFI), soit un indicateur de rapidité de déplacement du sang. Pour y arriver, il faut calculer le contraste et celui-ci est inversement proportionnel au BFI. Les modèles présentés comportent des paramètres expérimentaux importants, dont le paramètre β qu'on rencontre souvent dans la littérature des tavelures et qui est difficile à estimer.

— Zheng et Mertz [10 ; 9] expliquent, dans un premier temps, comment calculer directement le temps de corrélation d'une manière se rapprochant d'une approche sans modèle. On se base sur la théorie des tavelures pleinement développées et on intègre numériquement le contraste. Il y a toutefois les enjeux d'ergodicité : si les tavelures ne sont pas ergodiques, des corrections doivent être apportées, ajoutant une couche d'à priori. De plus, leur algorithme est relativement lourd, utilisant une approche pixel par pixel. Dans un second temps, ils présentent une manière de corriger le biais induit lors du calcul local du contraste spatial. On ajoute encore une couche de complexité et le besoin de savoir si les tavelures sont ergodiques.

Ces articles présentent des approches classiques, utilisant des méthodes statistiques.

Au cours des dernières années, les chercheurs ont utilisé de plus en plus l'apprentissage machine pour analyser les tavelures :

— Fredriksson et al. [2 ; 4] expliquent l'utilisation d'un réseau de neurones artificiels dans le contexte de l'imagerie de tavelures à différents temps d'acquisition. Ils y utilisent un réseau de type perceptron, pleinement connecté avec une seule couche cachée. Le modèle est fourni d'informations extraites d'images de tavelures. Différents réseaux sont initialisés aléatoirement et le meilleur est conservé pour le test. L'utilisation d'une fonction d'activation tanh est expliquée par le fait que les réseaux l'ayant sont meilleurs que ceux ayant d'autres fonctions d'activation. Ils arrivent à la conclusion que l'apprentissage machine permet aux tavelures laser d'être utilisées et améliorer l'interprétation des images.

— Kaler et al. [6] utilisent différentes approches d'apprentissage automatique pour la détection de graines infectées. Dans un premier temps, ils utilisent des approches plus classiques, comme des SVMs ou des KNNs, mais ensuite combinent des CNNs avec des LSTMs pour une analyse spatio-temporelle. Ils arrivent à la conclusion qu'un modèle à LSTM convolutionnel est meilleur que les autres en général. Cet article est particulièrement intéressant, car il compare plusieurs architecture et une méthode d'analyse robuste, soumettant les architectures à des données originales, sans bruit, puis ajoutant du bruit. De plus, parmis les autres articles, ils sont les seuls à considérer l'aspect spatial et temporel en même temps, profitant des CNNs et des RNNs. Par contre, il s'agit ici d'un problème de classification.

— Shang et al. [8] utilisent un CNN tridimensionnel pour analyser le flot sanguin à partir d'images de tavelures laser. Ils arrivent à des précisions supérieurs à 90%, mais font appel à une approche de classification, discrétisant le flot sanguin en plusieurs intervalles. Cet approche peut être avantageuse dans la prédiction, mais on perd de l'information qui est parfois très importante. Il faut aussi bien découper l'espace des résultat pour ne pas avoir un biais trop élevé pour certains intervalles. L'approche de CNN 3D reste toutefois très intéressante.

Chapitre 3

Choix des modèles

La programmation des modèles se fera à l'aide du module *PyTorch* du langage de programmation *Python*.

3.1 Modèle principal

Le modèle principal sélectionné est un mixte entre un réseau convolutionnel et un réseau récurrent, similaire à un modèle présenté dans [6]. Ce choix s'explique par la nature des tavelures et des statistiques importantes. Dans un premier temps, l'aspect spatial des tavelures est important à considérer. Comme il s'agit d'un motif d'interférence et d'imagerie, il est possible d'avoir des sections d'images qui sont sans information pertinente. Par exemple, des minimums sombres ou une absence d'illumination laissant des pixels sans intensité. L'avantage du CNN est aussi d'effectuer une sélection des régions importantes, diminuant la dimensionnalité. Il est important de noter que le pooling ne sera pas utilisé entre les couches de CNN, car cette étape diminue la qualité spatiale de l'information, parfois cachée. Il faut aussi mentionné que le réseau CNN ne devra pas être très profond étant donné que seules des statistiques de premier et de second ordre sont pertinente. Utiliser un réseau trop profond pourrait mener à l'extraction d'information non pertinente rendant l'apprentissage plus difficile. À la fin de l'étape de convolution, un mécanisme d'attention spatiale sera utilisé pour pondérer les images selon la pertinence des pixels à partir des caractéristiques extraites des couches de convolution.

Dans un second temps, un RNN de type GRU sera utilisé pour extraire l'information temporelle à partir de l'attention spatiale. Pour y arriver, un pooling sera appliqué pour vectoriser les caractéristiques. Par la suite un réseau de type GRU sera entraîné sur une séquence temporelle de tavelures pour extraire l'information scalaire du temps caractéristique. Le choix d'un GRU repose principalement sur sa simplicité contrairement à un LSTM. Il sera toujours possible de tester un LSTM pour vérifier les performances. L'avantage d'un réseau récurrent est dans sa capacité à se souvenir ou oublier l'information passée d'un séquence, ce qui est impor-

tant dans le cas des tavelures, car pour le temps caractéristique de corrélation, l'information réside dans toute la séquence temporelle et permettra d'estimer un scalaire en prédiction. Le nombre de couches sera petit pour éviter le surapprentissage : débuter avec une seule couche et augmenter au besoin.

3.2 Modèle secondaire

Si le temps le permet ou si le modèle principal s'avère problématique, le modèle secondaire est à étudier. Il s'agit d'un réseau récurrent convolutionnel, ConvRNN, de type ConvLSTM, comme utilisé dans [6]. Un avantage de cette approche est la possibilité d'extraire une carte locale du temps caractéristique de manière plus naturelle que le modèle précédent. Par contre, l'interprétation du modèle reste un peu plus difficile, demandant plus de paramètres, et pourrait être plus long à l'entraînement. De plus, *PyTorch* ne permet pas explicitement de créer un réseau ayant des couches ConvRRN, que ce soit ConvLSTM ou autre. Il existe toutefois une page GitHub où ConvLSTM est construit sur les bases de *PyTorch*, voir le lien `https://github.com/ndrplz/ConvLSTM_pytorch`, ayant plus de 2000 étoiles et 444 *forks*.

3.3 Pipeline d'entraînement

Les données utilisées pour l'entraînement sont présentées au chapitre 4. Les données générées peuvent être sous plusieurs formats, mais on va se contenter à les laisser en *ndarray* de *NumPy*, facilement transférable à *PyTorch*. Si besoin, on peut sauvegrader les données au format *.npy* et les stocker sur un disque. *PyTorch* permet d'utiliser des utilitaires de création de datasets d'entraînement, de validation et de tests, ce qui sera utilisé pour s'assurer d'avoir un entraînement en *batch* avec un certain aléatoire pour diversifier le contenu de chaque *batch*. Il serait aussi possible à l'aide de l'orienté objet de faire en sorte que la génération des données d'entraînement soient *on the go*, c'est-à-dire que quand on veut accéder à une *batch*, on la génère directement. Cela peut aider à garder la mémoire disque libre, mais empêche la reproductibilité facile des entraînements.

Il faut aussi penser à la normalisation. L'échelle d'intensité des images peut changer d'une expérience à l'autre, notamment à cause de l'intensité du laser, de la qualité de la caméra, etc. Dans le cas du projet courant, les images sont par défaut toutes à la même échelle, mais on peut induire une différence sans trop de problème. Il est donc important que le réseau normalise sur une même échelle les intensités. Deux approches sont souvent utilisées : z-score ou normalisation $[0, 1]$. La première approche est intéressante, car elle centre et réduit l'intensité, ayant alors une moyenne similaire à 0 et une variance proche de 1. Or, dans plusieurs cas d'imagerie, l'intensité décroît exponentiellement plus on s'éloigne du centre, comme montré à la figure 3.1. Avec le z-score, un biais peut être induit selon la proportion de l'image qui est

plus foncée que le reste à cause de la perte d'intensité. On peut penser à une approche par noyau où on applique le z-score localement dans des petites régions des images, mais cela aussi induit un biais vers l'uniformisation des intensités selon la taille des tavelures. Une normalisation z-score temporelle peut aussi être intéressante à première vue, mais dans certains cas, les tavelures sont temporellement stationnaire (comme dans ce projet), donc on uniformise une fois de plus les images et on perd de l'information cruciale. On doit donc se pencher vers la normalisation $[0, 1]$. On force alors les pixels à avoir une échelle identique, sans vraiment changer les probabilités. Elles se retrouvent certes translatées et modulées, mais leur nature ne devrait pas changer.

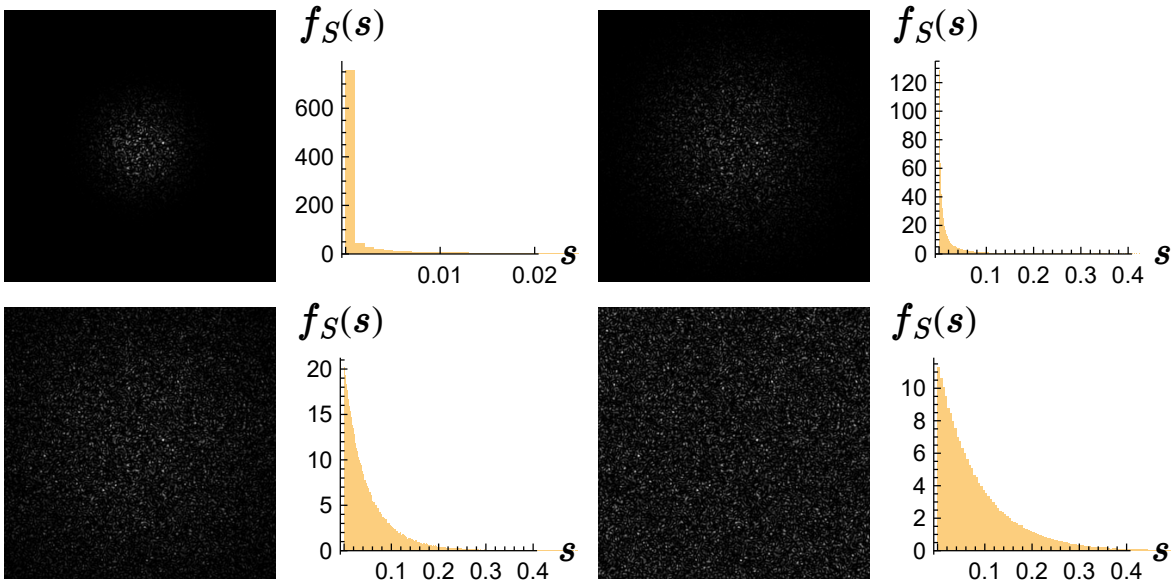

FIGURE 3.1 – Simulations de tavelures pleinement développées où un masque gaussien est appliqué sur l'intensité résultante, montrant l'effet d'un faisceau gaussien sur la tache laser. On montre que plus la tache laser est petite sur l'image, plus l'intensité est biaisée vers 0. De telles images doivent être normalisée judicieusement. Image tirée de [3].

Le modèle principal est schématiquement présenté à la figure 3.2, avec les hyperparamètres initiaux. Ceux-ci peuvent changer durant le développement du projet. Pour l'entraînement, plusieurs modalités de simulations seront présentées aléatoirement à chaque batch. Les modalités pour chaque séquence seront décrites dans un fichier csv de métadonnées. Par modalité, on fait référence au paramètres physiques de l'imagerie, comme le temps caractéristique, le temps d'intégration et la fonction de corrélation. Comme ces paramètres jouent un rôle important dans l'entraînement, plusieurs instances de chaque combinaisons seront requises. Le DataLoader permet de bien distribuer aléatoirement les diverses simulations pour que le réseau aie une vue d'ensemble et non seulement quelques exemples ayant les même propriétés.

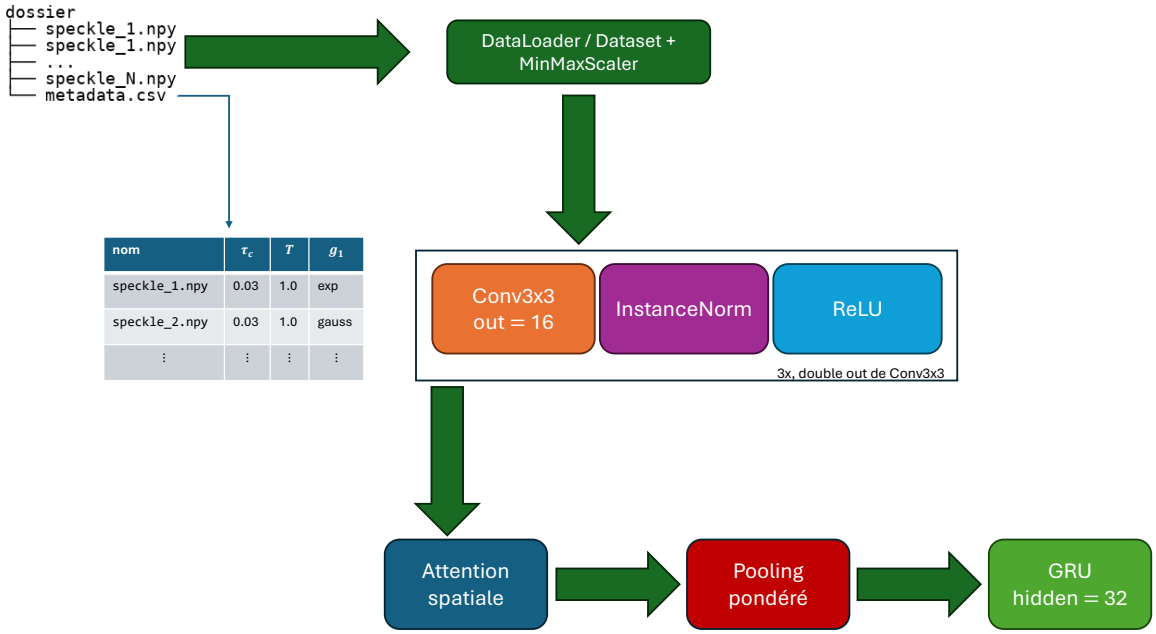

FIGURE 3.2 – Schéma montrant le pipeline du réseau utilisé pour le projet. Les hyperparamètres peuvent changer au cours du développement.

Chapitre 4

Données du modèle

Les données d'entraînement du modèle seront générées numériquement selon un algorithme précis. De cette manière, on peut modifier les paramètres statistiques des tavelures comme on veut. De plus, une grande quantité de données peuvent être générées dans une fraction du temps que demande un montage expérimental. L'algorithme est basé principalement sur [5] où on présente une façon de simuler l'intégration temporelle des tavelures. Toutefois, cet algorithme ne simule pas de séries temporelles. Initialement, on génère les phaseurs selon l'Algorithme 1.

Algorithme 1 Simulation de tavelures pleinement développées intégrées temporellement

Entrées : $N \in \mathbb{N}^*$, $R \in \mathbb{R}_+^*$, $N_T \in \mathbb{N}^*$, $\Delta T \in \mathbb{R}_+^*$, $g : \mathbb{R} \to \mathbb{R}$
Sorties : I avec $I \in \mathbb{R}_+^{N \times N}$

```
Générer Θ ∈ [−π, π]^{N×N} avec Θ_ik ∼ U(−π, π) indépendants
Définir Σ ∈ {0, 1}^{N×N} comme un disque centré de rayon R
T ← [0, ΔT, 2ΔT, ..., (N_T − 1)ΔT]
I ← 0_{N×N}
G ← 0_{N_T×N_T}
Pour i = 1 à N_T faire
    Pour k = 1 à N_T faire
        G_ik ← g(|T_i − T_j|)
    Fin Pour
Fin Pour
λ ← eigvals(G)/N_T                    ▷ eigvals calcule les valeurs propres
Pour n = 1 à N_T faire
    A ← exp{jΘ}                       ▷ Exponentielle terme à terme
    A ← F_2D(A)√λ_n                   ▷ Transformée de Fourier 2D
    A ← Σ ⊙ A
    A ← F_2D^{−1}(A)                  ▷ Transformée de Fourier inverse 2D
    A_ℝ ← |A|                         ▷ Module terme à terme
    I ← I + A_ℝ^2                     ▷ Carré terme à terme
Fin Pour
```

Pour une série temporelle, sans intégration temporelle, on doit modifier la création des amplitudes complexes pour ajouter une dimension temporelle ayant une certaine corrélation, comme montré à l'Algorithme 2.

Algorithme 2 Simulation temporelle de tavelures pleinement développées

Entrées : $N \in \mathbb{N}^*$, $R \in \mathbb{R}_+^*$, $P \in \mathbb{N}^*$, $C \in \mathbb{R}^{P\times P}$
Sorties : $(A_\mathbb{R}, I)$ avec $A_\mathbb{R} \in \mathbb{R}^{P\times N\times N}$ et $I \in \mathbb{R}_+^{P\times N\times N}$

Générer $\mathcal{R} \in \mathbb{R}^{P\times N\times N}$ avec $\mathcal{R}_{pik} \sim \mathcal{N}\left(0, \frac{1}{2}\right)$ et $\rho\left(\mathcal{R}_{pik}, \mathcal{R}_{p'i'k'}\right) = \begin{cases} C_{pp'} & \text{si } (i,k) = (i',k') \\ 0 & \text{sinon} \end{cases}$

Générer $\mathcal{I} \in \mathbb{R}^{P\times N\times N}$ avec $\mathcal{I}_{pik} \sim \mathcal{N}\left(0, \frac{1}{2}\right)$ et $\rho\left(\mathcal{I}_{pik}, \mathcal{I}_{p'i'k'}\right) = \begin{cases} C_{pp'} & \text{si } (i,k) = (i',k') \\ 0 & \text{sinon} \end{cases}$

Définir $\Sigma \in \{0,1\}^{N\times N}$ comme un disque centré de rayon R
$A \leftarrow \mathbf{0}_{P\times N\times N}$
Pour $p = 1$ **à** P **faire**
  $A_p \leftarrow \mathcal{R}_p + j\mathcal{I}_p$
  $A_p \leftarrow \mathcal{F}_{2D}(A_p)$
  $A_p \leftarrow \Sigma \odot A_p$
  $A_p \leftarrow \mathcal{F}_{2D}^{-1}(A_p)$
Fin Pour
$A_\mathbb{R} \leftarrow |A|$
$I \leftarrow A_\mathbb{R}^2$

La corrélation provenant de la matrice de corrélation C est conservée dans le champ réel $A_\mathbb{R}$ et la corrélation de l'intensité I en est simplement le carré. Il est possible de mettre en commun l'Algorithme 1 et l'Algorithme 2 : on génère les amplitudes selon l'Algorithme 2, permettant ainsi une corrélation temporelle spécifique, puis ensuite on se base sur l'Algorithme 1. On obtient l'Algorithme 3. Un avantage de cet algorithme est qu'il génère des séries temporelles stationnaires grâce à la création des séries temporelles multinormales, contrairement à d'autres algorithmes trouvés dans la littérature. On peut alors décider de la corrélation temporelle, tant qu'elle donne une matrice de corrélation valide, c'est-à-dire positive (semi) définie. On peut aussi moduler le temps d'intégration simulée, donnant des images plus ou moins floues. Un désavantage de cette approche est sa complexité en mémoire. En effet, on doit générer $N_T \times P \times N \times N$ amplitudes complexes, même si au final on perd la première dimension à l'aide d'une réduction. L'interdépendance des P champs $N \times N$ fait aussi en sorte qu'on ne peut pas les générer *on the go*, on doit les créer toutes en même temps pour que la dépendance temporelle soit cohérente. Il est toutefois possible de vectoriser l'algorithme pour tirer avantage des architectures GPU. La première étape de vectorisation se trouve dans la boucle interne finale sur P, ainsi les opérations internes peuvent se faire en bloc. Une deuxième étape est d'aussi vectoriser la boucle externe finale sur N_T, mais on multiplie la mémoire requise en une passe. Soit on a beaucoup de VRAM à consacrer, soit on doit avoir de petites simulations. En se limitant par contre à la boucle interne, des GPUs de moyenne gamme peuvent très bien gérer des simulations intéressantes. On peut aussi s'assurer de libérer l'espace dès que possible

ou encore de restreindre la précision des flottants, par exemple à 32 bits. De cette manuère, les amplitudes complexes sont sur 64 bits, ce qui est suffisant en général.

Algorithme 3 Simulation temporelle de tavelures pleinement développées intégrées temporellement

Entrées : $N \in \mathbb{N}^*$, $R \in \mathbb{R}_+^*$, $N_T \in \mathbb{N}^*$, $\Delta T \in \mathbb{R}_+^*$, $g : \mathbb{R} \to \mathbb{R}$, $P \in \mathbb{N}^*$
Sorties : I avec $I \in \mathbb{R}_+^{P \times N \times N}$

```
Définir Σ ∈ {0,1}^{N×N} comme un disque centré de rayon R
T ← [0, ΔT, 2ΔT, ..., (N_T − 1)ΔT]
I ← 0_{P×N×N}
G ← 0_{N_T×N_T}
c ← 0_P
Pour p = 0 à P − 1 faire
    c_p ← g(p · (N_T − 1)ΔT)
Fin Pour
C ← Toeplitz(c)                    ▷ Construit une matrice Toeplitz avec première ligne c
Pour i = 1 à N_T faire
    Pour k = 1 à N_T faire
        G_ik ← g(|T_i − T_j|)
    Fin Pour
Fin Pour
λ ← eigvals(G)/N_T
Pour n = 1 à N_T faire
    Générer R ∈ ℝ^{P×N×N}, R_pik ∼ N(0, 1/2) et ρ(R_pik, R_p'i'k') = { C_pp'  si (i,k) = (i',k')
                                                                      { 0      sinon
    Générer I ∈ ℝ^{P×N×N}, I_pik ∼ N(0, 1/2) et ρ(I_pik, I_p'i'k') = { C_pp'  si (i,k) = (i',k')
                                                                      { 0      sinon
    Pour p = 1 à P faire
        A_p ← R_p + jI_p
        A_p ← F_2D(A_p)√λ_n
        A_p ← Σ ⊙ A_p
        A_p ← F_2D^{−1}(A_p)
        A_ℝ ← |A_p|
        I_p ← I_p + A_ℝ^2
    Fin Pour
Fin Pour
```

La génération des données se fera à l'aide du langage de programmation *Python*, en utilisant les puissantes librairies *NumPy* et *SciPy* principalement. De plus, pour l'accélération GPU, la librairie *CuPy* sera aussi importante, étant essentiellement un portage de *NumPy* (et d'un peu de *SciPy*) sur les cartes graphiques compatibles CUDA [1]. Pour plus d'information sur le module, visiter le site web `https://cupy.dev/`.

Si le temps le permet, il serait aussi intéressant de tester différentes variantes, notamment l'ajout de bruit, la non uniformité de la tache laser et la variation spatiale du temps caractéristique de corrélation. Les deux premiers cas sont relativement triviaux, alors que le dernier requiert une refonte de l'algorithme à l'étape de création des amplitudes complexes et lors de la pondération par les valeurs propres. Pour l'instant, seules les tache uniforme sans bruit et de temps de corrélation stationnaire dans l'espace sont considérés. La figure **??** montre trois simulations d'une même séquence où le temps d'intégration est de 0.3 seconde et le temps caractéristique est de 1 seconde. La corrélation de la séquence au complet est présentée à la figure **??**. À la figure **??**, on montre trois simulations aussi d'une même séquence, mais cette fois le temps d'intégration est de 1 seconde pour un temps caractéristique de 0.3 seconde. La corrélation de la séquence au complet est présentée à la figure **??**. À noter la différence de la courbe. Dans le premier cas, la corrélation est exponentielle, alors que dans le second cas elle est gaussienne. Ce genre d'images seront générées pour différents paramètres et serviront à l'entraînement, la validation et le test.

1. Aussi disponible pour les architectures ROCm, mais à l'état expérimental.

Chapitre 5

Tâches à accomplir

Annexe A

Base mathématique des tavelures

Bibliographie

[1] David Briers, Donald D. Duncan, Evan Hirst, Sean J. Kirkpatrick, Marcus Larsson, Wiendelt Steenbergen, Tomas Stromberg et Oliver B. Thompson : Laser speckle contrast imaging : theoretical and practical limitations. *Journal of Biomedical Optics*, 18(6) :066018, juin 2013.

[2] Ingemar Fredriksson, Martin Hultman, Tomas Strömberg et Marcus Larsson : Machine learning in multiexposure laser speckle contrast imaging can replace conventional laser Doppler flowmetry. *Journal of Biomedical Optics*, 24(01) :1, janvier 2019.

[3] Gabriel Genest : Exploration théorique et simulations des tavelures laser dans un contexte d'imagerie, 2025.

[4] Martin Hultman, Marcus Larsson, Tomas Strömberg et Ingemar Fredriksson : Speed-resolved perfusion imaging using multi-exposure laser speckle contrast imaging and machine learning. *Journal of Biomedical Optics*, 28(03), mars 2023.

[5] Edward James, Samuel Powell et Peter Munro : Simulation of statistically accurate time-integrated dynamic speckle patterns in biomedical optics. *Optics Letters*, 46(17) : 4390, septembre 2021.

[6] Nikhil Kaler, Vimal Bhatia et Amit Kumar Mishra : Deep Learning-Based Robust Analysis of Laser Bio-Speckle Data for Detection of Fungal-Infected Soybean Seeds. *IEEE Access*, 11 :89331–89348, 2023.

[7] Chang Liu, Kıvılcım Kiliç, Sefik Evren Erdener, David A. Boas et Dmitry D. Postnov : Choosing a model for laser speckle contrast imaging. *Biomedical Optics Express*, 12(6) :3571, juin 2021.

[8] Kai Jing Shang, Yuan Yuan, Hong Li Liu, Ren Bing Wang, Wei Nan Gao, Yong Bi et Yang Yu : Estimation of absolute wide-range blood flow by deep learning-based laser speckle contrast imaging. *Optics and Lasers in Engineering*, 193 :109056, octobre 2025.

[9] Shuqi Zheng et Jerome Mertz : Correcting sampling bias in speckle contrast imaging. *Optics Letters*, 47(24) :6333, décembre 2022.

[10] Shuqi Zheng et Jerome Mertz : Direct characterization of tissue dynamics with laser speckle contrast imaging. *Biomedical Optics Express*, 13(8) :4118, août 2022.